



Nottingham Trent
University

Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 5: Containerisation & Distributed Technologies for CI Workflows

Dr Pedro Machado pedro.machado@ntu.ac.uk



Nottingham Trent
University

Department of Computer Science

Lecture Outline

In this lecture will cover:

- Motivation for modern CI
- Containerisation fundamentals
- Docker in CI pipelines
- Scalable distributed CI
- Kubernetes-based orchestration
- Observability and reliability
- Agentic AI in CI
- Security and governance

This lecture contributes to the following learning outcomes:

MLO2: Critically evaluate the strengths and weaknesses of different computational intelligence techniques for solving complex problems.

MLO4: Understand the ethical implications of using computational intelligence technologies and the potential impact on society.

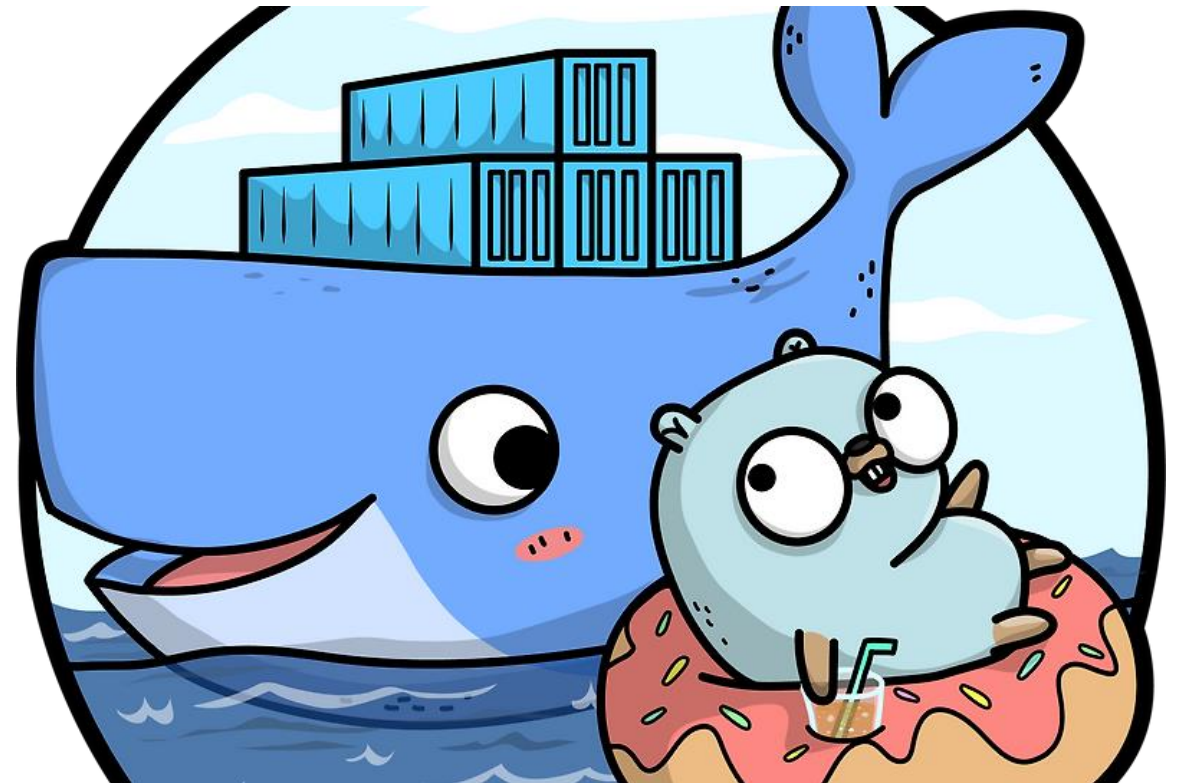
MLO5: Apply computational intelligence techniques to solve real-world problems, demonstrating practical problem-solving skills.

MLO6: Develop proficiency in programming and using computational tools for implementing and evaluating intelligent systems.

MLO8: Communicate effectively the concepts, methodologies, and results of computational intelligence research to both technical and non-technical audiences.

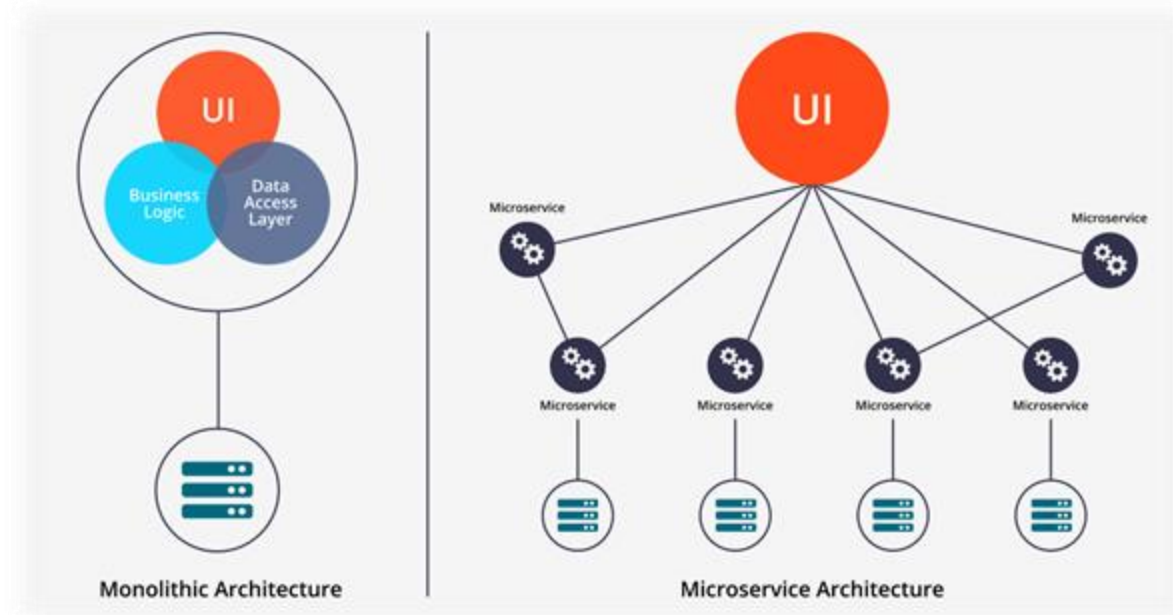
Motivation for containerised technology

- Modern software systems combine multiple services, libraries, runtimes, and deployment targets, increasing integration and environmental complexity.
- Development and CI environments frequently differ, leading to build and test failures that are not caused by code changes.
- Reproducible builds ensure identical artefacts are produced from the same source and configuration, regardless of execution context.
- Reproducible tests ensure test results reflect software behaviour rather than environmental variation.
- Containerisation mitigates these issues by providing consistent, controlled execution environments across the entire CI pipeline.



From monolithic to pipelines

- Traditional monolithic build processes rely on manual or semi-manual integration steps, where code is built and tested infrequently on shared environments.
- Such approaches delay feedback, making defects harder to detect and increasing the cost of fixing integration issues.
- Automated CI pipelines decompose the build process into repeatable stages such as build, test, and packaging, executed on every change.
- Rapid, automated feedback improves development velocity by allowing teams to identify and resolve issues early.
- Consistent, automated testing improves software quality by enforcing standards and reducing human error throughout the integration process.



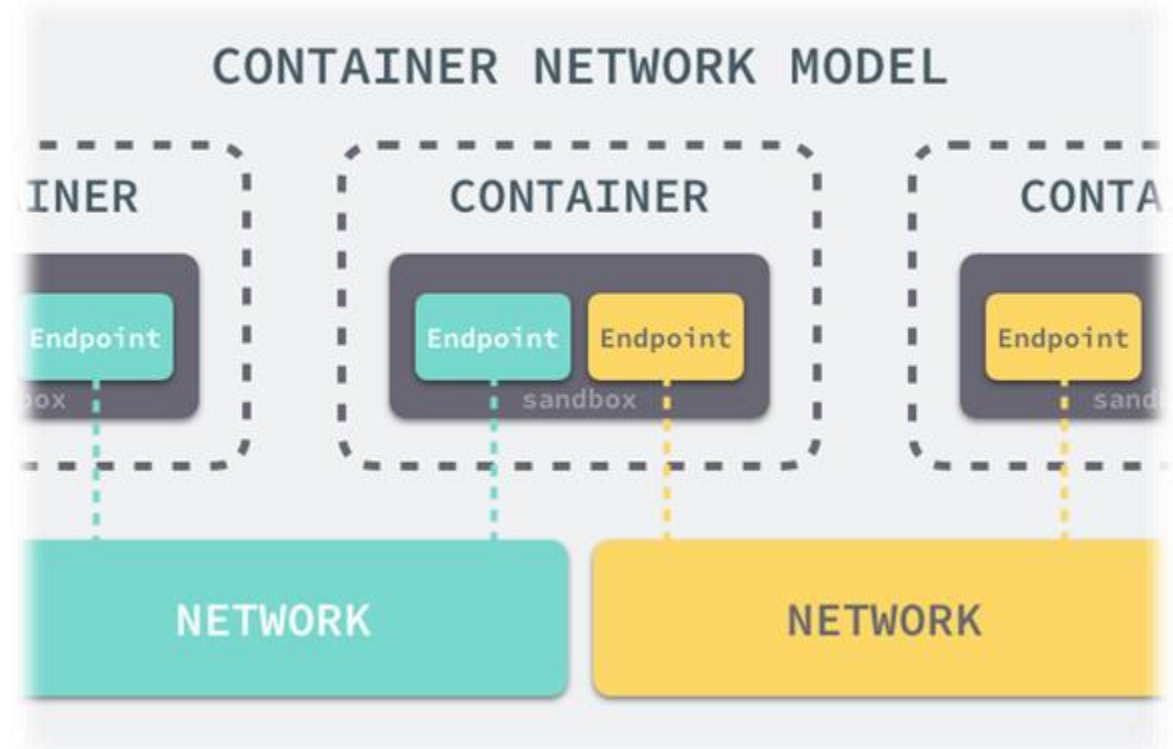
Core problem

- Environment drift occurs when development, testing, and CI environments gradually diverge due to differences in operating systems, dependencies, or configuration changes.
- Such drift leads to non-deterministic behaviour, where software passes tests in one context but fails in another.
- Scaling CI increases complexity by introducing multiple runners, parallel jobs, and heterogeneous execution environments.
- Without strong isolation and standardisation, CI systems become fragile and difficult to debug at scale.
- Containerisation provides a practical solution by enforcing consistent environments while enabling CI workloads to scale reliably across distributed infrastructure.



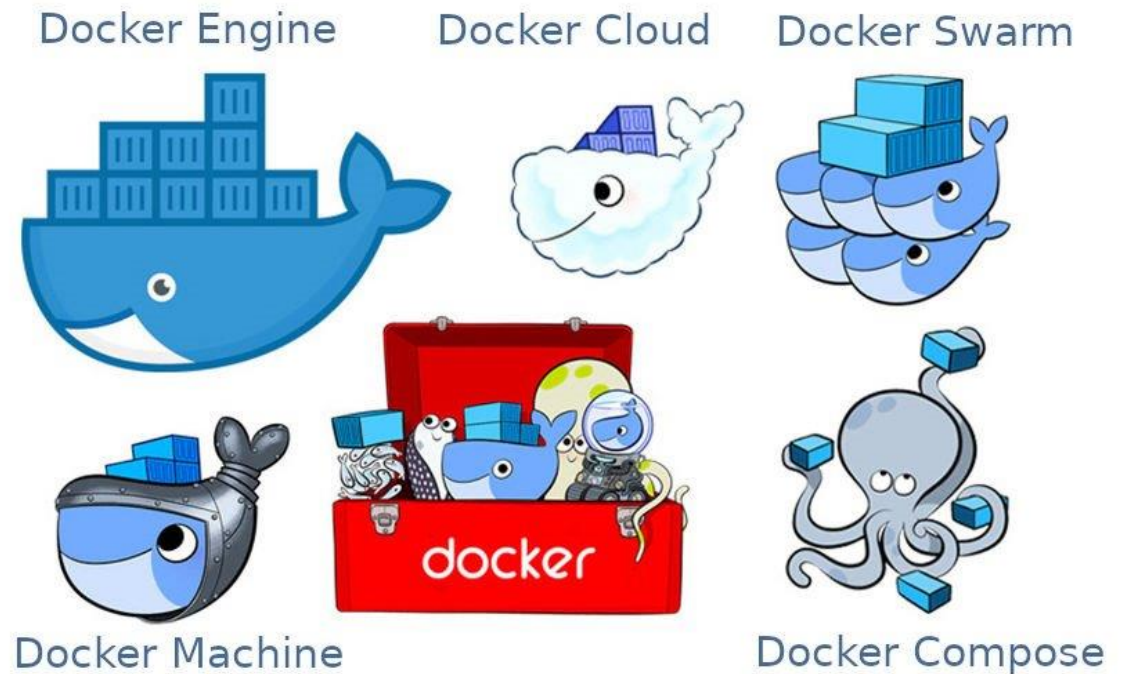
Why containers for CI?

- Containers provide consistent execution environments across all CI stages, including build, test, and packaging, eliminating discrepancies between development and integration systems.
- Identical container images can be reused throughout the pipeline, ensuring that software is validated under the same conditions in which it is built.
- Containers start quickly and incur minimal overhead compared to virtual machines, enabling rapid job execution.
- Fast provisioning allows CI systems to create and destroy isolated environments on demand for each pipeline run.
- Such characteristics support scalable, parallel CI workflows while maintaining reliability and reproducibility.

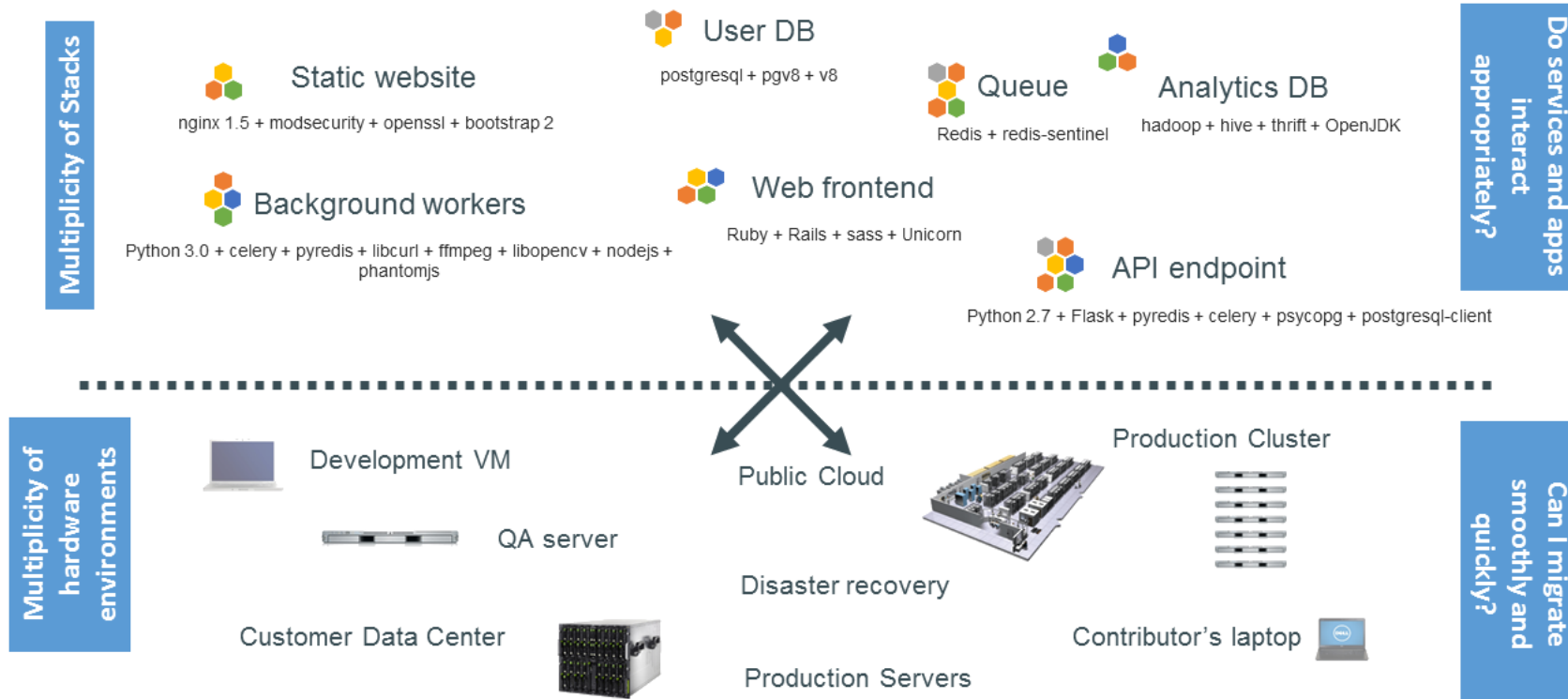


What is Docker ?

- Open platform for developers and sysadmins to build, ship and run distributed applications
- Can run on popular cross platforms (amd64, x86, arm64, armhf, etc.)
- Supported by the wide majority of cloud platforms including Amazon EC2, Google Compute Engine, Microsoft Azure and Rackspace.



The Challenge



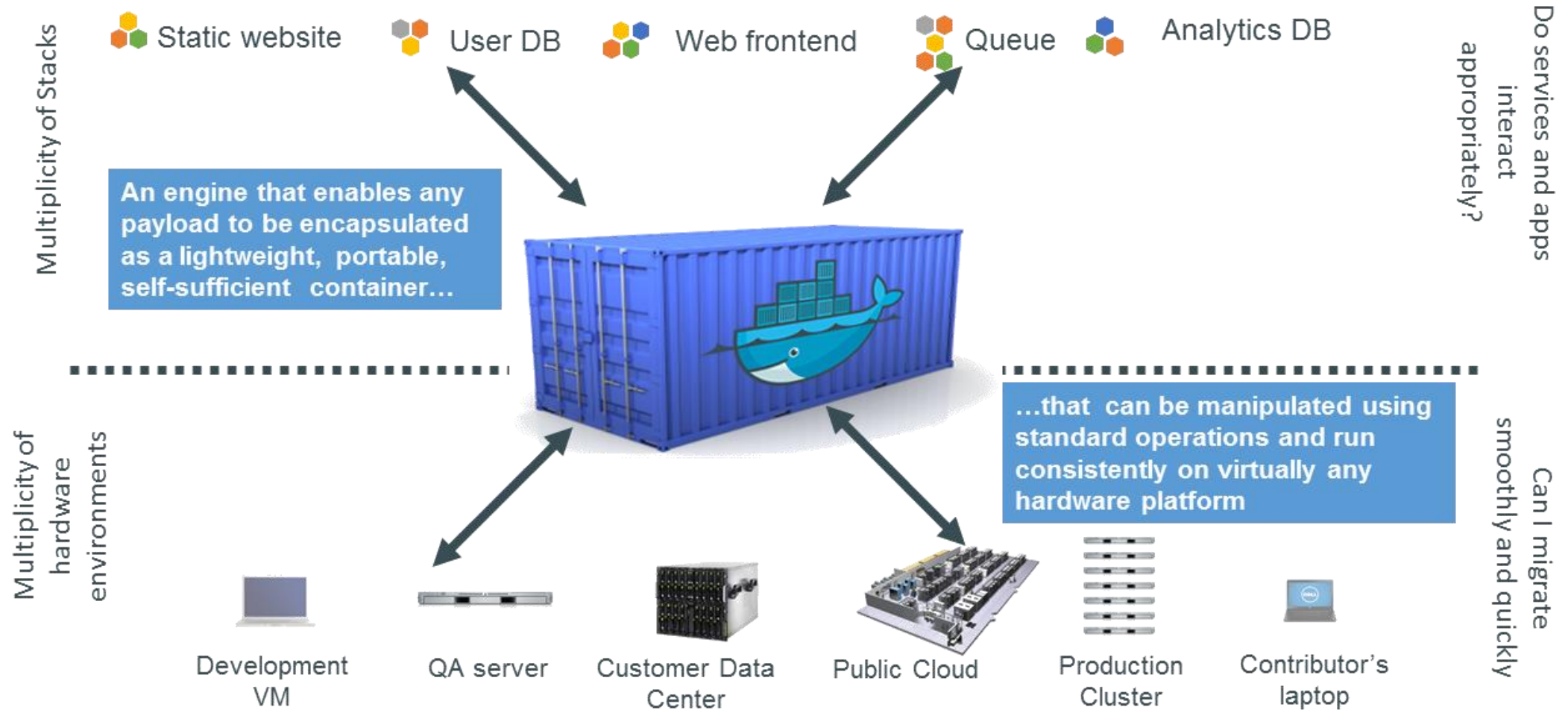
What and where?

	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
							

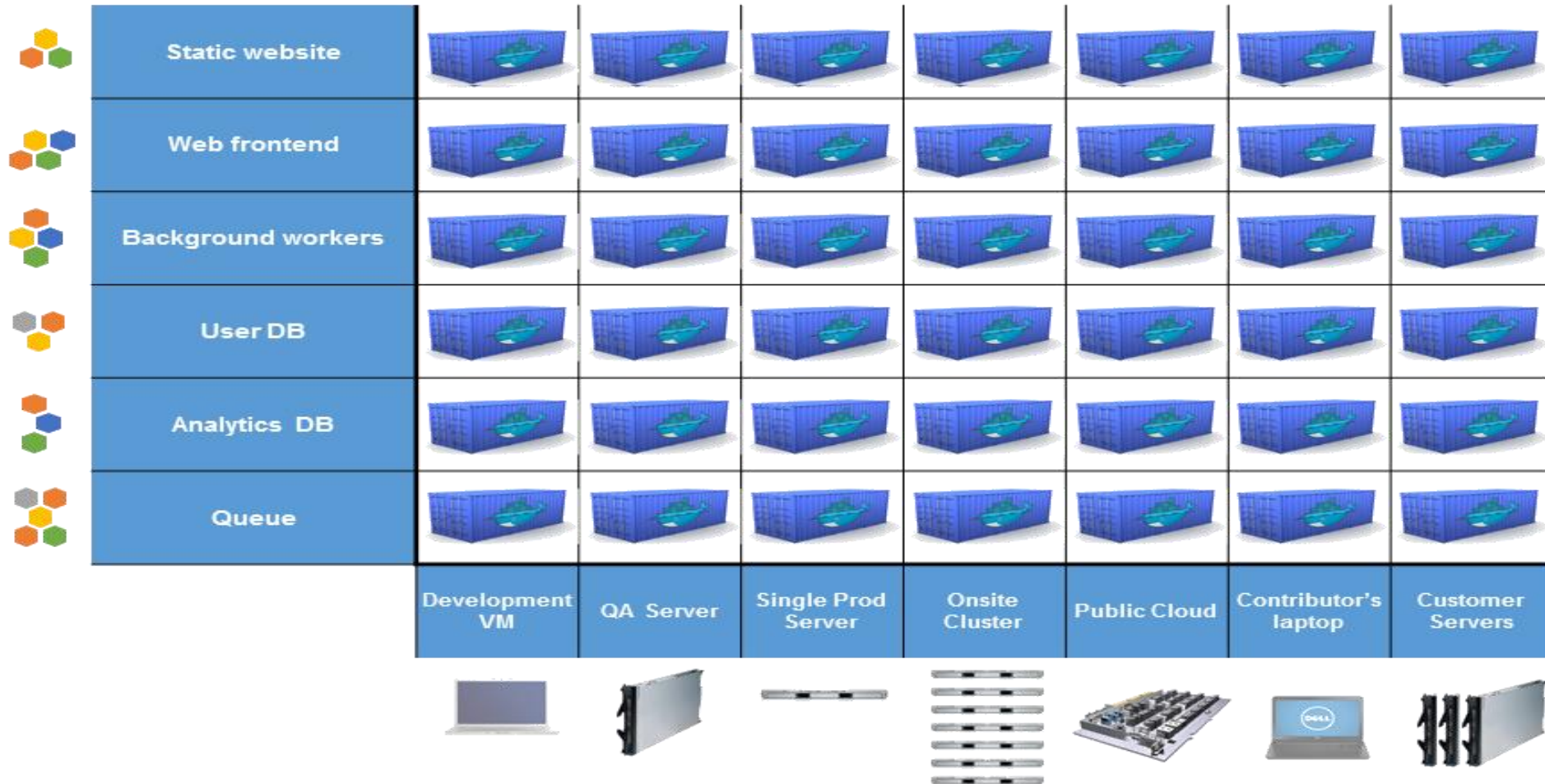
Solution: Intermodal Shipping Container



Docker is a Container System for Code.....

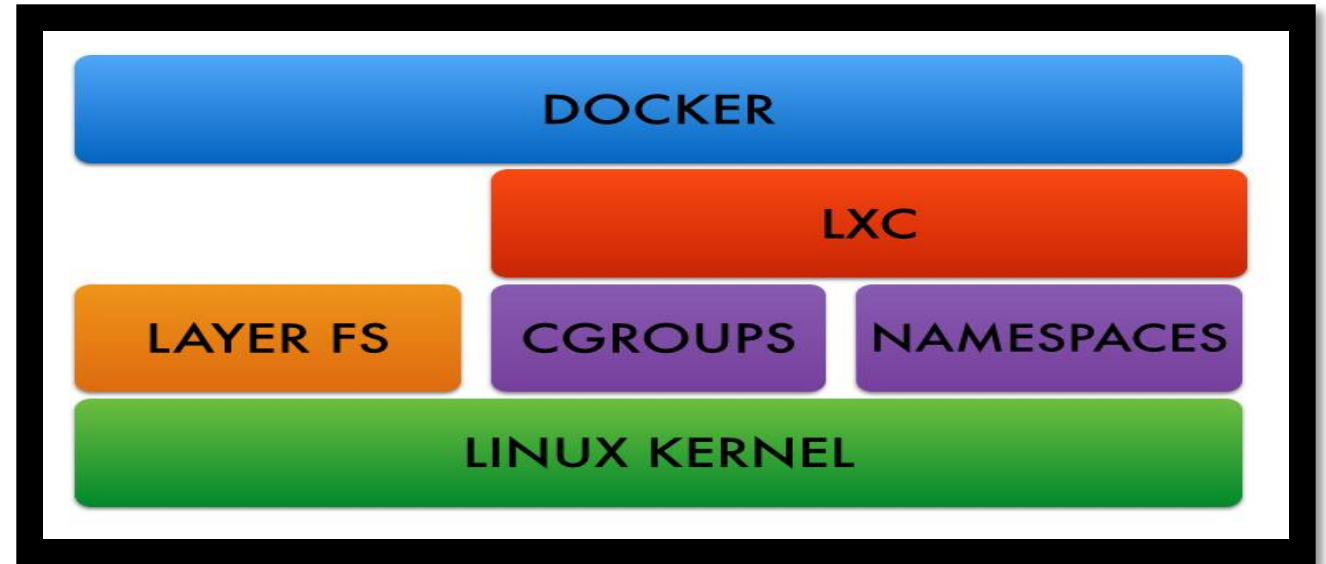


Docker provides the answer to what and where



Docker Architecture

- Docker Engine
 - CLI
 - Docker Daemon
 - Docker Registry
- Docker Hub
 - Cloud service
 - Share Applications
 - Automate workflows
 - Assemble apps from components
- Docker images
- Docker containers

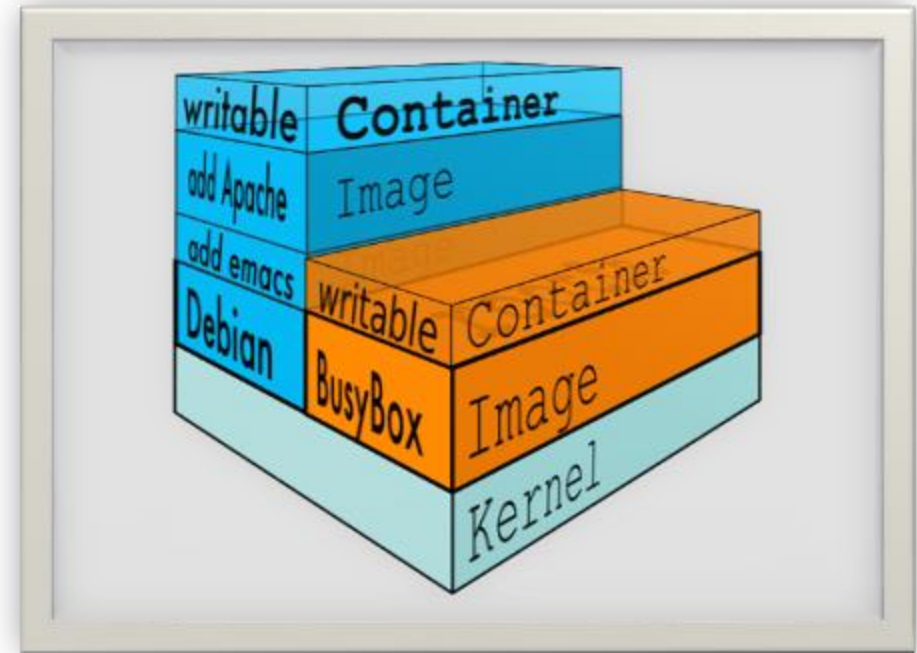


Docker Containers...

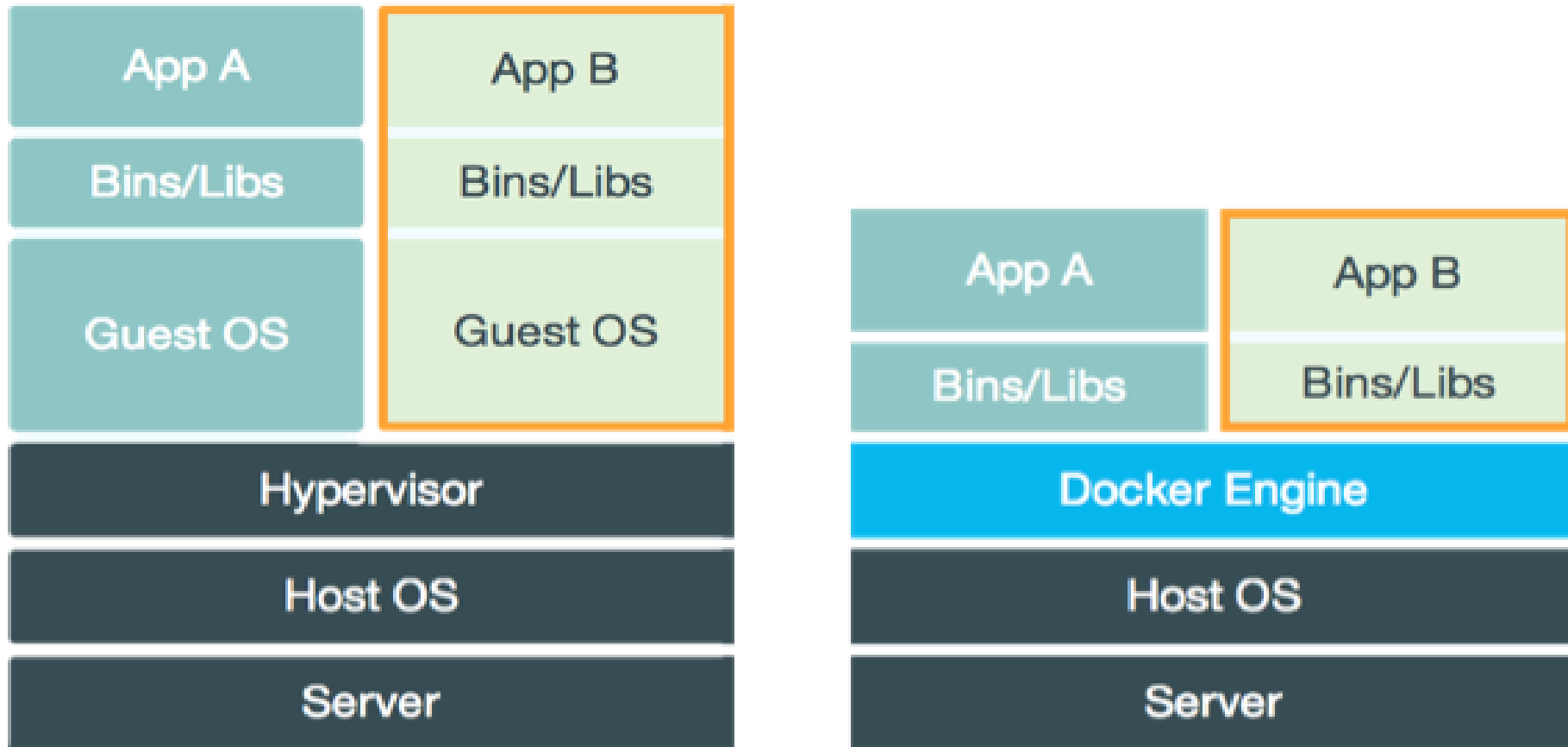
Units of software delivery (ship it!)

- run everywhere
 - regardless of kernel version
 - regardless of host distro
 - (but container and host architecture must match*)
- run anything
 - if it can run on the host, it can run in the container
 - i.e., if it can run on a Linux kernel, it can run

*Unless you emulate CPU with qemu and binfmt

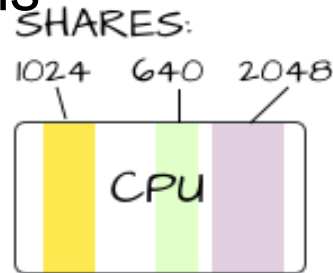


Virtual Machine Versus Container.....



Linux Cgroups

- Kernel Feature
- Groups of processes
- Control resource allocations
 - CPU
 - Memory
 - Disk
 - I/O
- May be nested



CGROUP #1

Gets half as much CPU time as cgroup #3.

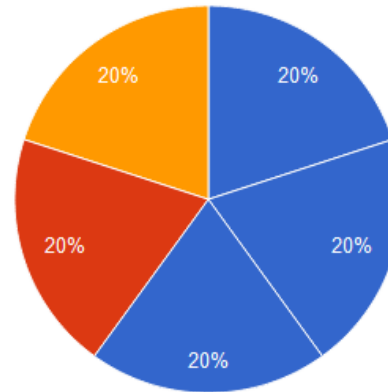
CGROUP #2

Gets the least CPU time.

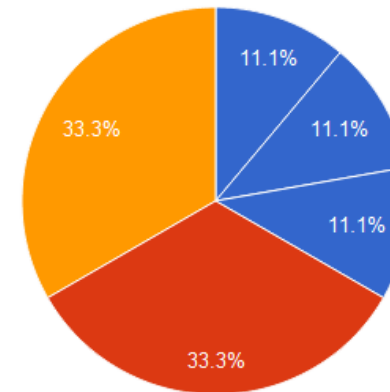
CGROUP #3

Gets the most CPU time.

CPU usage per process without cgroups



CPU usage per process with cgroups



■ User A - process 1
■ User A - process 2
■ User A - process 3
■ User B - process 4
■ User C - process 5

Implications for CI

- Containers reduce startup and execution overhead compared to virtual machines, leading to faster build and test jobs.
- Shorter job execution times enable quicker feedback to developers, improving development cadence and productivity.
- Lightweight container isolation allows multiple CI jobs to run concurrently on the same host without significant performance penalties.
- Higher runner density improves infrastructure utilisation and reduces the cost of operating CI systems at scale.
- Such efficiencies make it feasible to scale CI pipelines horizontally while maintaining predictable performance and isolation.



Containerised CI stages

- CI pipelines use dedicated build containers to compile code and resolve dependencies within a controlled environment.
- Build containers ensure that compiler versions, libraries, and tooling remain consistent across pipeline executions.
- Test containers execute automated tests against the build artefacts in isolated, reproducible environments.
- Separation of build and test containers reduces coupling and improves fault isolation within the pipeline.
- Containerised stages enable parallel execution and simplify scaling across distributed CI infrastructure.

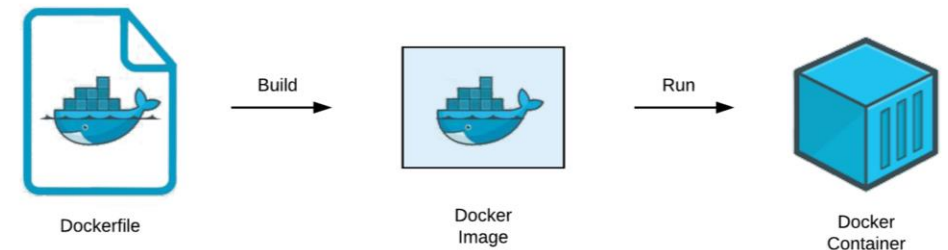
Dockerfile

- Like a Makefile (shell script with keywords)
- Extends from a Base Image
- Results in a new Docker Image
- Imperative, not Declarative
- A Docker file lists the steps needed to build an images
- docker build is used to run a Docker file
- Can define default command for docker run, ports to expose, etc

```
# syntax=docker/dockerf
FROM ubuntu:18.04
COPY . /app
RUN make /app
CMD python /app/app.py
```

Dockerfiles in CI

- Dockerfiles act as executable specifications that precisely describe how build and test environments are constructed.
- Each instruction documents tooling, dependencies, and configuration required by the CI pipeline.
- Storing Dockerfiles in version control ensures that environment changes are tracked alongside application code.
- Version-controlled environments support traceability, allowing historical builds and tests to be reproduced reliably.
- Dockerfiles enable CI systems to automatically and consistently recreate environments on demand.



Dependency isolation

- Containers and VMs isolate application dependencies, preventing conflicts between libraries and tools used by different projects or CI jobs.
- Isolation avoids shared state on CI hosts, reducing interference between concurrent pipeline executions.
- Each CI job executes with a clean, predictable environment defined by its container image.
- Deterministic builds ensure that identical inputs always produce the same outputs.
- Such predictability simplifies debugging and increases confidence in CI results.

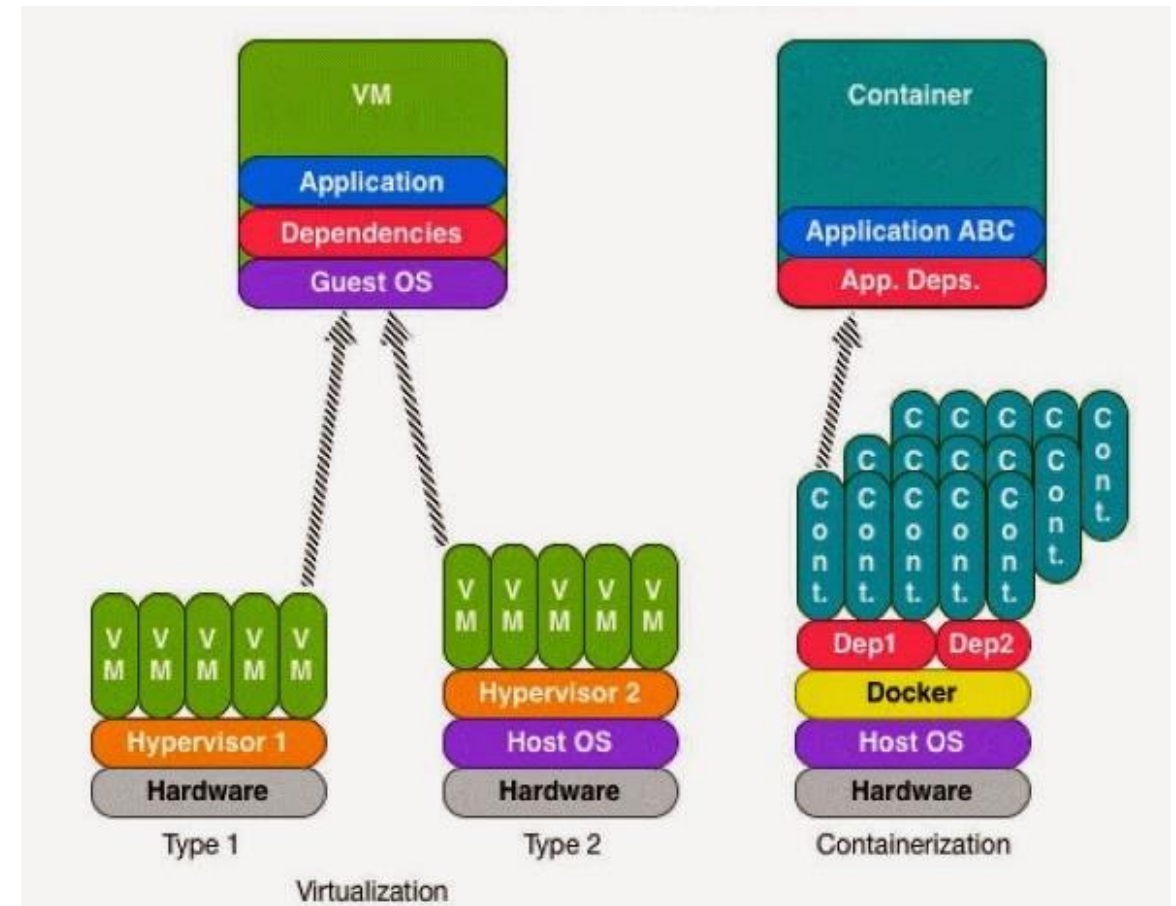


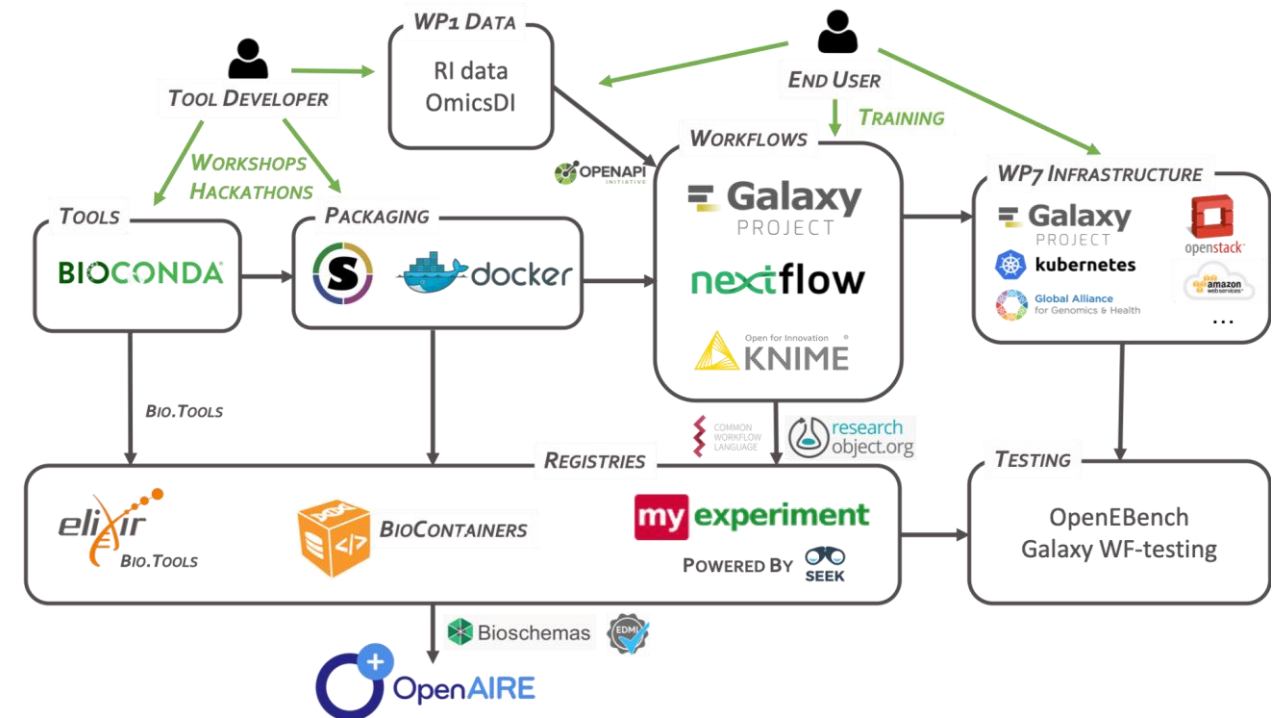
Image tagging and versioning

- Container images are identified using tags that represent versions, configurations, or build states.
- Semantic tags encode meaning, such as software version, build type, or target environment.
- Consistent tagging strategies allow CI pipelines to select the correct image for each stage.
- Image tags enable traceability by linking deployed artefacts back to specific commits and CI runs.
- Effective versioning supports rollback, auditing, and long-term reproducibility of CI workflows.



Container Registries

- Container registries store and distribute container images used throughout CI pipelines.
- Public registries provide broad access to base images and shared tools, while private registries offer controlled access for proprietary or sensitive artefacts.
- Private registries improve security by limiting image distribution and enforcing organisational policies.
- Registries act as caches, reducing build times by reusing previously built image layers.
- Effective registry use improves CI performance, scalability, and reliability across distributed environments.



Security in containers

- CI pipelines rely on base images that must be sourced from trusted and well-maintained repositories.
- Unverified or outdated base images can introduce vulnerabilities into the build and test process.
- Vulnerability scanning identifies known security issues in container images and their dependencies.
- Integrating scanning into CI enables early detection and remediation of security risks.
- Secure image practices improve overall pipeline integrity and reduce downstream exposure.



Building containers workflow

A typical CI-oriented docker buildx command sequence (build, cache, tag, push) is:

1) Build + push a single-arch image (common for CI runners)

```
docker buildx build --file Dockerfile --tag REGISTRY_HOST/ORG/APP:sha-${GIT_SHA} -  
-tag REGISTRY_HOST/ORG/APP:latest --cache-from  
type=registry,ref=REGISTRY_HOST/ORG/APP:buildcache --cache-  
to type=registry,ref=REGISTRY_HOST/ORG/APP:buildcache,mode=max --push .
```

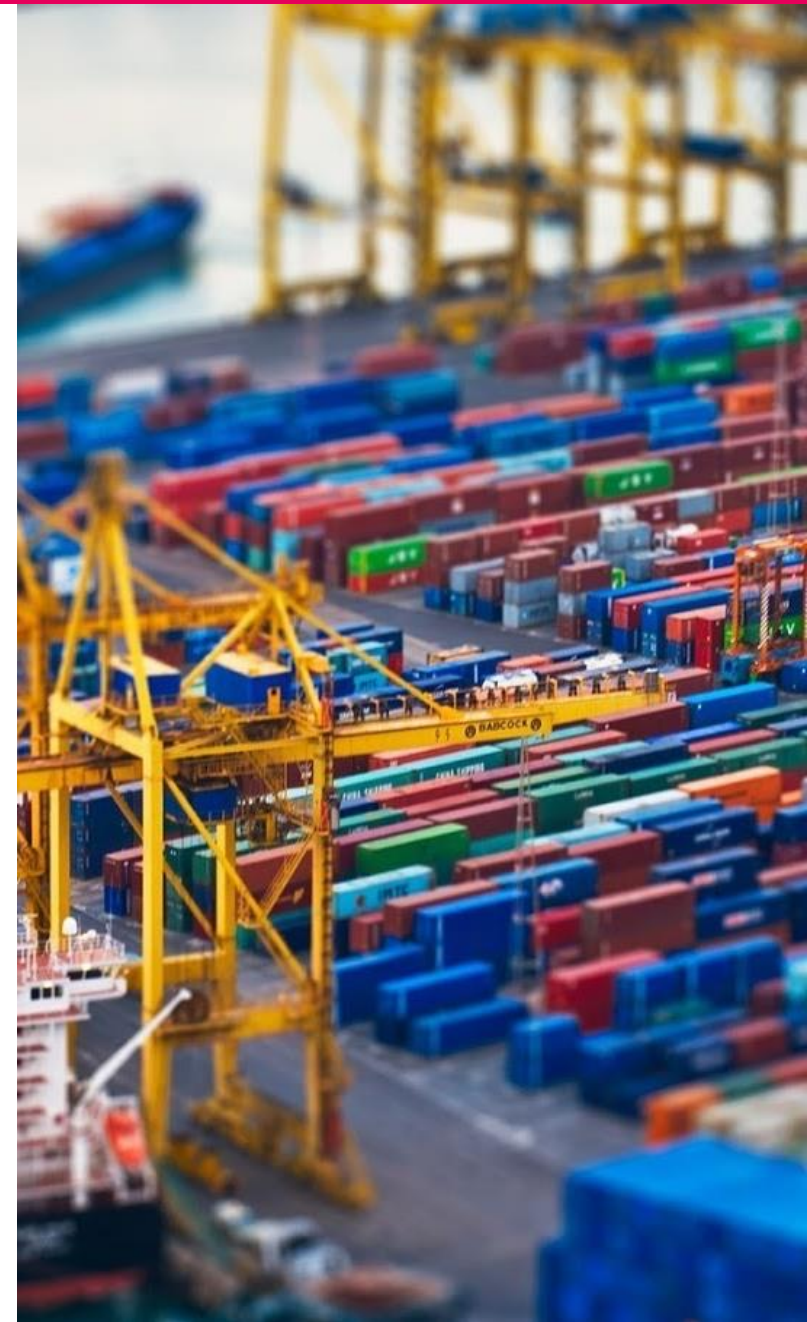
2) Build + push a multi-arch image (if you target amd64 + arm64)

```
docker buildx build --platform linux/amd64,linux/arm64 --file Dockerfile --tag  
REGISTRY_HOST/ORG/APP:sha-${GIT_SHA} --tag REGISTRY_HOST/ORG/APP:release-${VERSION}  
--cache-from type=registry,ref=REGISTRY_HOST/ORG/APP:buildcache --  
cache to type=registry,ref=REGISTRY_HOST/ORG/APP:buildcache,mode=max --push .
```



Scaling CI

- CI scaling addresses increased workload by running multiple pipeline jobs in parallel rather than sequentially.
- Parallel jobs reduce end-to-end pipeline time by splitting work across independent stages (for example, unit tests, integration tests, and static analysis).
- Distributed runners execute CI jobs across multiple machines or nodes, avoiding bottlenecks on a single host.
- Runner distribution improves resilience by allowing workloads to continue even if individual runner nodes fail.
- Effective scaling requires consistent environments, scheduling, and resource controls to maintain predictable performance across runners.

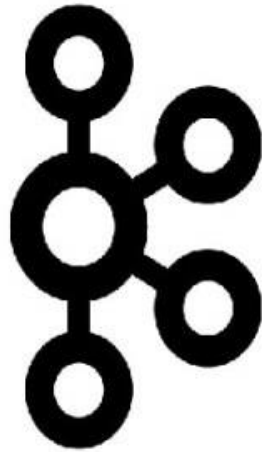
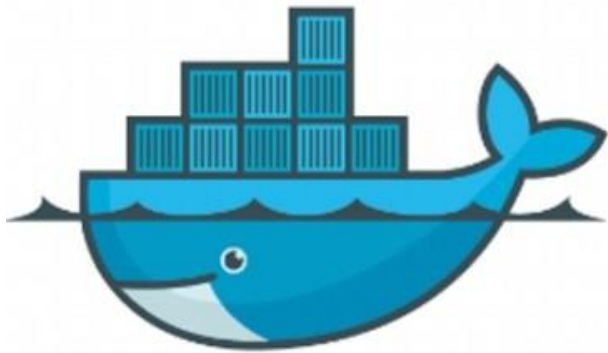


Containers as CI workers

- Containers act as lightweight CI workers that execute individual jobs within isolated environments.
- Ephemeral execution ensures that each container is created for a single job and destroyed immediately after completion.
- Clean state per job prevents contamination from previous builds, tests, or cached artefacts.
- Isolation improves reliability by ensuring that concurrent jobs do not interfere with one another.
- Such a model supports scalable, repeatable, and fault-tolerant CI execution across distributed infrastructure.



Distributed CI architectures

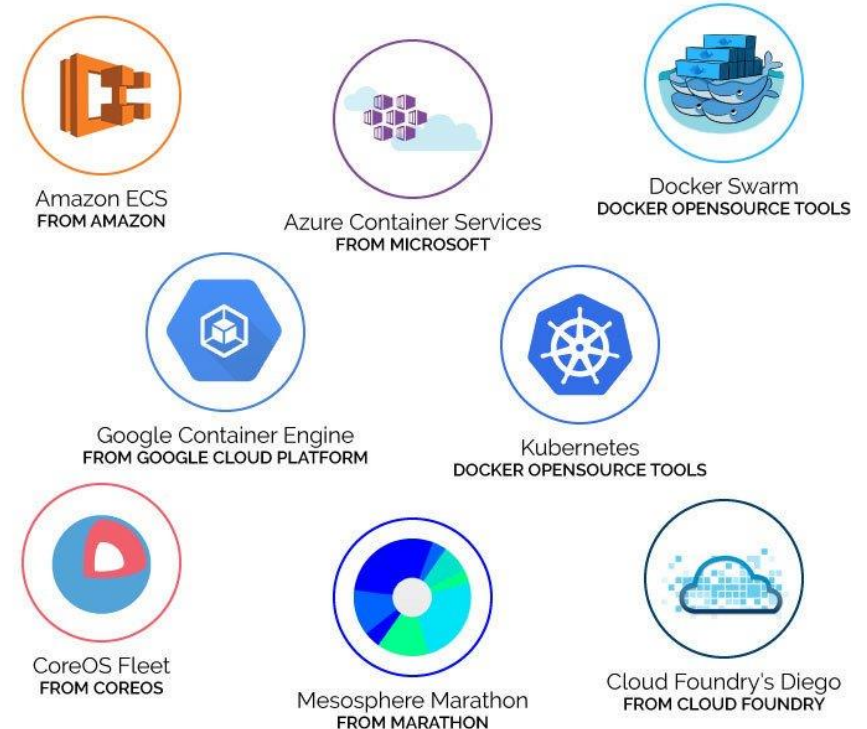


- Distributed CI systems separate coordination logic from execution logic.
- A central controller manages pipelines, schedules jobs, and tracks execution state.
- Worker nodes execute CI jobs in isolated environments, typically using containers.
- Decoupling control and execution enables horizontal scaling by adding or removing workers as demand changes.
- Such architectures improve throughput, resilience, and manageability of large-scale CI systems.

Orchestration concepts

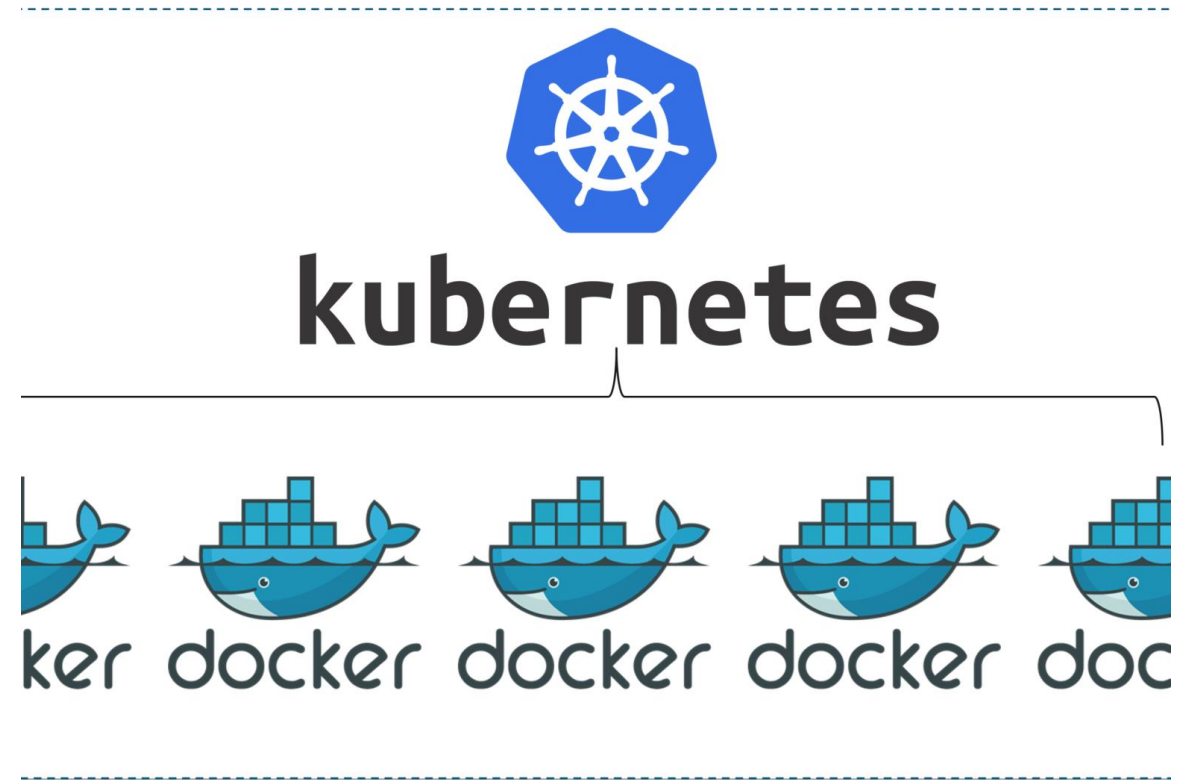
- Orchestration systems automate the management of containerised workloads across multiple nodes.
- Scheduling determines where and when CI jobs are executed based on availability and constraints.
- Resource allocation assigns CPU, memory, and other resources to each job to prevent contention.
- Policies ensure fair sharing while respecting priorities and limits.
- Effective orchestration is essential for predictable performance in large-scale, container-based CI environments.

Tools of Container Orchestration

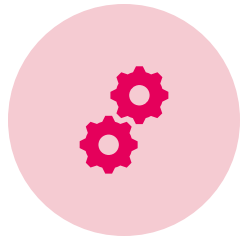


Kubernetes overview

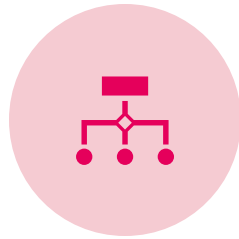
- Kubernetes (K8s) is a container orchestration platform designed to manage containerised workloads at scale.
- Pods are the smallest deployable units, encapsulating one or more containers that share networking and storage.
- Nodes are physical or virtual machines that run pods and provide compute resources.
- Clusters consist of multiple nodes managed as a single logical system.
- Such abstractions enable scalable, resilient execution of CI workloads across distributed infrastructure.



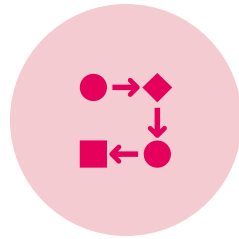
CI on Kubernetes



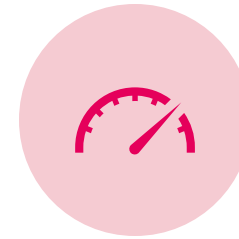
CI SYSTEMS DEPLOY
RUNNERS AS
CONTAINERISED
WORKLOADS MANAGED
BY K8S.



RUNNERS ARE
PROVISIONED
DYNAMICALLY IN
RESPONSE TO INCOMING
PIPELINE JOBS.



AUTOMATIC SCALING
CREATES ADDITIONAL
RUNNERS DURING PEAK
DEMAND AND REMOVES
THEM WHEN IDLE.

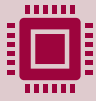


DYNAMIC PROVISIONING
ENSURES EFFICIENT USE
OF CLUSTER
RESOURCES WHILE
MAINTAINING FAST JOB
START TIMES.



KUBERNETES ENABLES
FAULT TOLERANCE BY
RESCHEDULING CI
RUNNERS IF NODES FAIL.

Resource management



CPU and memory limits constrain CI jobs to predefined resource budgets, preventing individual workloads from monopolising shared infrastructure.



Resource requests inform the scheduler of minimum requirements, enabling predictable placement of CI runners.



GPU sharing allows accelerator resources to be allocated selectively to jobs requiring hardware acceleration, such as machine learning tests.



Controlled GPU access avoids contention and enables efficient utilisation across multiple CI workloads.



Fair sharing policies balance resource distribution across users and pipelines, maintaining performance and system stability at scale.

Observability challenges

- Observability in CI environments requires visibility into system behaviour across multiple abstraction levels.
- Logs capture discrete events and execution details generated by CI jobs and containers.
- Metrics provide quantitative measurements such as resource usage, job duration, and failure rates.
- Traces follow the execution path of jobs across distributed components and services.
- Correlating logs, metrics, and traces is essential for diagnosing failures and performance issues in distributed CI systems.



Monitoring containers

- Container monitoring distinguishes between host-level metrics and container-level metrics.
- Host metrics provide a global view of CPU, memory, disk, network, and GPU utilisation across the system.
- Container metrics attribute resource consumption to individual CI jobs and workloads.
- GPU monitoring tracks accelerator usage, memory consumption, and execution time for GPU-enabled containers.
- Correlating host, container, and GPU metrics enables accurate performance analysis and capacity planning in CI environments.



Failure modes

- Distributed CI systems must tolerate failures at both infrastructure and workload levels.
- Node failure can interrupt running jobs when a worker becomes unavailable or unreachable.
- Orchestration platforms detect such failures and reschedule affected jobs on healthy nodes.
- Job retries allow transient failures, such as network issues or resource contention, to be recovered automatically.
- Proper failure handling improves reliability while avoiding unnecessary pipeline interruptions.



Why AI in CI?



CI pipelines have grown increasingly complex, incorporating numerous stages, tools, and conditional workflows.



Manual configuration and rule-based automation struggle to scale with this complexity.



Static automation lacks the ability to adapt to changing codebases, workloads, and failure patterns.



Artificial intelligence enables data-driven decision-making based on historical pipeline behaviour.



AI-assisted CI can optimise execution, prioritise tests, and improve reliability beyond traditional automation.

Agentic AI

- Agentic AI refers to systems composed of autonomous agents capable of making decisions and taking actions to achieve defined goals.
- An agent observes its environment, reasons about possible actions, and selects actions based on objectives and constraints.
- Decision-making is driven by feedback, allowing agents to adapt their behaviour over time.
- Agents operate with varying degrees of autonomy, from advisory roles to fully independent execution.
- In CI contexts, agentic AI enables pipelines to respond intelligently to changing conditions rather than following fixed rules.



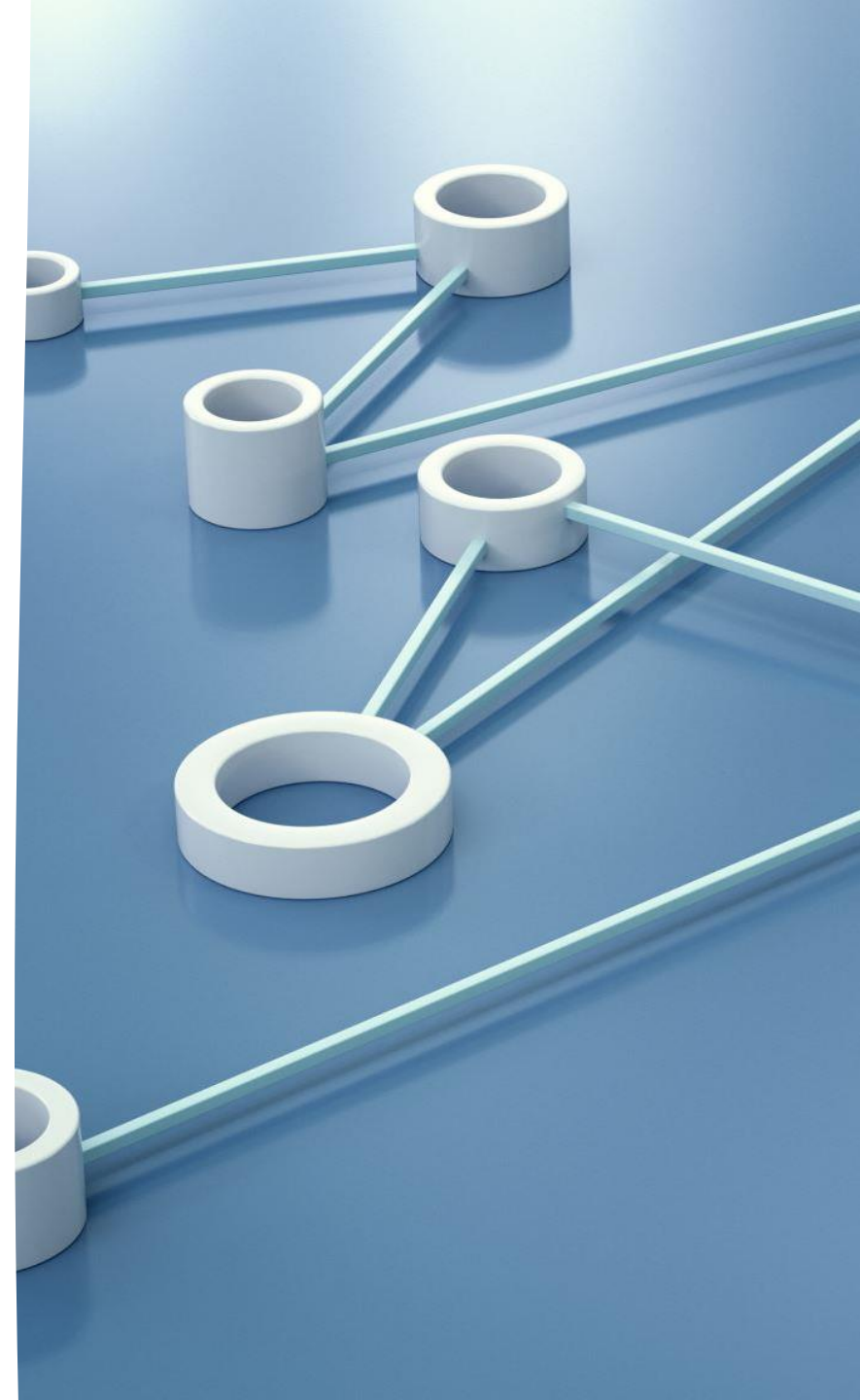
Agent roles in CI

- Agentic AI can assume specialised roles within CI pipelines.
- A pipeline optimiser adjusts execution order, parallelism, and resource allocation to reduce overall runtime.
- Test selector agents choose relevant subsets of tests based on code changes and historical results.
- Selective testing reduces execution time while maintaining confidence in correctness.
- Such role-based agents enhance efficiency without requiring manual pipeline redesign.



Agentic workflows

- Agentic workflows integrate autonomous agents directly into CI pipelines.
- Feedback loops allow agents to observe pipeline outcomes such as failures, durations, and resource usage.
- Continuous adaptation enables agents to modify decisions based on observed performance and trends.
- Over time, workflows evolve to better match project characteristics and operational constraints.
- Such adaptability distinguishes agentic CI from static, rule-based automation.



Human-in-the-loop CI



- Human-in-the-loop CI combines autonomous agent decision-making with human oversight.
- Agents propose actions or optimisations based on observed pipeline behaviour.
- Human operators review, approve, or override agent decisions when necessary.
- Oversight ensures accountability, safety, and alignment with organisational policies.
- Such collaboration balances automation benefits with human judgement and control.

Why sandboxing matters?

- Sandboxing constrains the execution environment of CI workloads and autonomous agents.
- Safety is achieved by preventing unauthorised access to host systems, networks, and sensitive data.
- Predictability ensures that actions produce consistent and bounded effects.
- Sandboxes limit the impact of faulty code, misconfiguration, or unexpected agent behaviour.
- Such constraints are essential for trustworthy automation in CI environments.



Threat models

- Threat modelling identifies potential risks arising from untrusted or faulty workloads in CI environments.
- Malicious code may attempt to escape isolation, access sensitive data, or disrupt infrastructure during pipeline execution.
- Runaway agents may consume excessive resources or trigger unintended actions due to flawed logic or feedback loops.
- Distributed CI systems amplify impact if such behaviour is not properly contained.
- Explicit threat models inform the design of isolation, monitoring, and control mechanisms for secure CI operation.



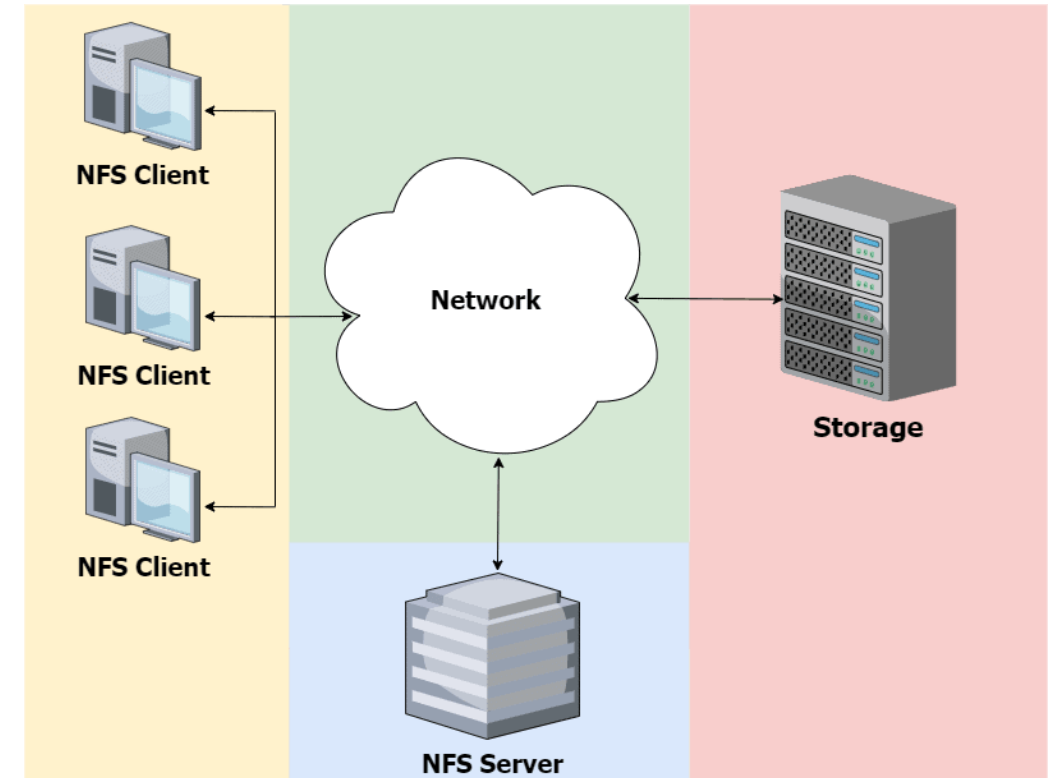
**THREAT
MODELING
CAPABILITIES**

Resource limits

- Resource limits define strict boundaries on CPU and GPU memory, and execution time for CI jobs and agents.
- CPU and memory limits prevent individual workloads from degrading overall system performance.
- GPU memory limits ensure fair access to accelerators and avoid resource starvation.
- Time budgets bound execution duration, preventing stalled or runaway jobs.
- Enforced limits improve predictability, stability, and cost control in large-scale CI environments.

Network and filesystem control

- Network controls restrict outbound and inbound connectivity for CI jobs and agents.
- Limiting network access reduces the risk of data exfiltration and unintended external interactions.
- Filesystem controls constrain read and write access to only required directories and artefacts.
- Least-privilege execution ensures workloads operate with the minimum permissions necessary to complete their tasks.
- Such controls reduce attack surface and improve security and predictability in CI pipelines.



Determinism in agent runs

- Determinism ensures that agent behaviour is consistent given the same inputs and environment.
- Reproducible agent runs allow CI outcomes to be recreated for verification and auditing.
- Controlled randomness and fixed configurations reduce non-deterministic behaviour.
- Deterministic execution simplifies debugging by enabling precise reproduction of failures.
- Such properties are essential for trusting autonomous agents in CI workflows.

Failure containment



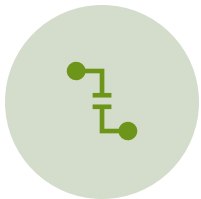
Failure containment limits the impact of faults within CI pipelines and agentic workflows.



Isolation ensures that failures remain confined to individual jobs or containers.



Blast radius reduction prevents cascading failures across runners, nodes, or pipelines.



Controlled retries and fallbacks enable recovery without widespread disruption.



Effective containment improves overall system resilience and operational stability.

Governance and ethics

- Governance defines rules and responsibilities for the use of automation in CI systems.
- Accountability ensures that actions taken by agents can be traced to responsible stakeholders.
- Clear ownership and audit trails support compliance and post-incident analysis.
- Trust in automation depends on transparency, predictability, and demonstrable reliability.
- Ethical deployment requires balancing efficiency gains with control, safety, and human oversight.



Secure CI by design

- Secure CI systems are designed with security considerations integrated from the outset.
- Defence in depth applies multiple, overlapping security controls across infrastructure, containers, and pipelines.
- Layered protections reduce reliance on any single mechanism or assumption.
- Continuous monitoring and validation strengthen long-term security posture.
- Such design principles are essential for resilient and trustworthy CI environments.



Putting it all together



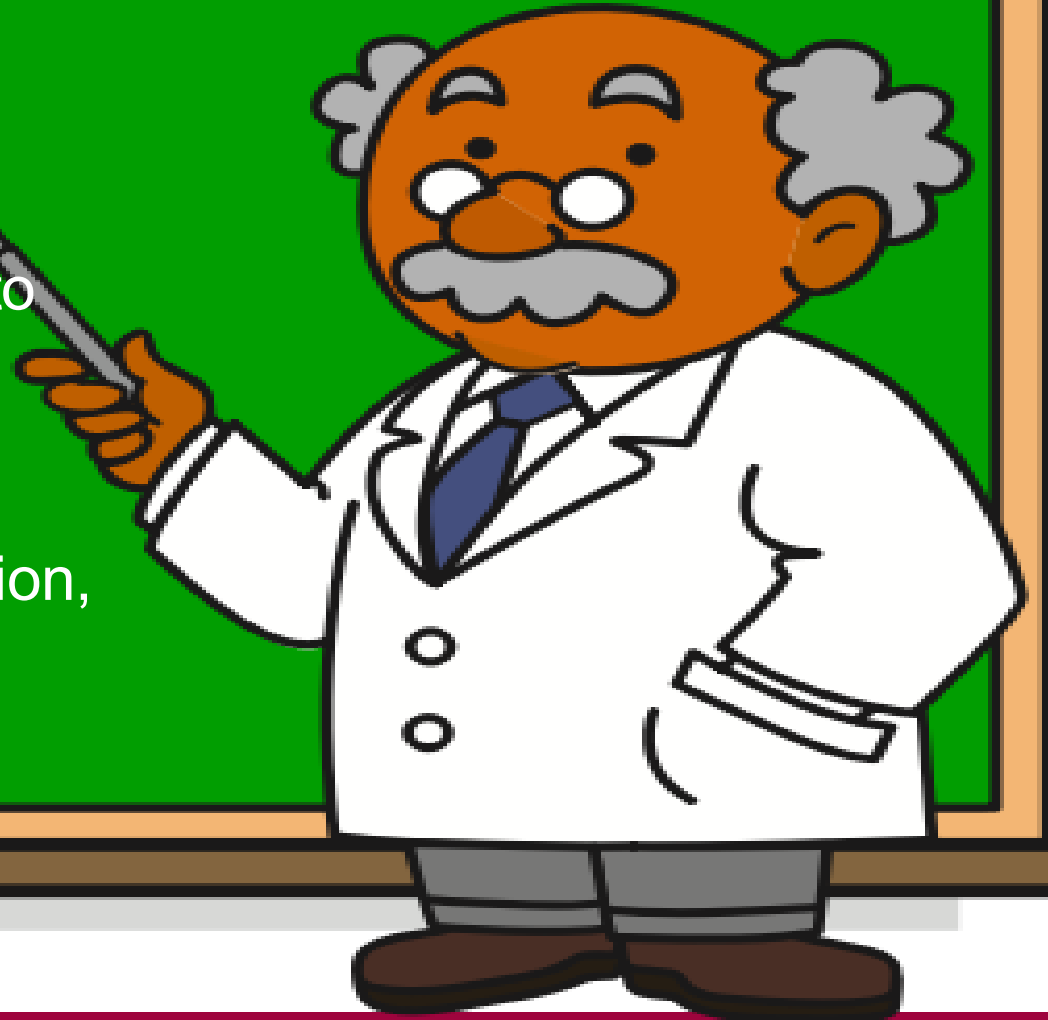
- Docker provides consistent, isolated execution environments for CI workloads.
- CI pipelines automate build, test, and packaging processes across distributed infrastructure.
- AI agents introduce adaptive decision-making into pipeline execution and optimisation.
- Combined, these technologies enable scalable, reproducible, and intelligent CI systems.
- Integration requires careful design to balance automation, safety, and control.

Open challenges

- Operating large-scale CI systems incurs significant infrastructure and operational costs.
- Balancing automation benefits with cost efficiency remains an ongoing challenge.
- Safety concerns arise from increased autonomy and complex interactions between components.
- Verifying the correctness and reliability of agent decisions is non-trivial.
- Addressing these challenges is critical for sustainable adoption of advanced CI technologies.

Key takeaways

- Containers enable scalable and reproducible CI by providing consistent, isolated execution environments.
- Container-based pipelines support efficient parallelism and high runner density.
- Agentic AI introduces adaptive intelligence into CI workflows.
- Agents enable dynamic optimisation beyond static, rule-based automation.
- Effective integration depends on strong isolation, observability, and governance.



Quiz (1/3)

Q1. What is the main cause of non-deterministic CI failures?

- A. Environment drift
- B. Insufficient logging
- C. Poor test coverage
- D. Slow networks

Q2. What is the primary purpose of a Dockerfile in CI?

- A. Monitoring containers
- B. Storing images remotely
- C. Scheduling jobs
- D. Defining build environments

Quiz (2/3)

Q3. Why do containers enable higher CI runner density?

- A. Stronger security guarantees
- B. Built-in orchestration
- C. Lightweight resource usage
- D. Persistent shared storage

Q4. What component decides where CI jobs run in a distributed system?

- A. Worker node
- B. Container image
- C. Registry
- D. Scheduler

Quiz (3/3)

Q5. Which task is best suited to an AI test selector agent?

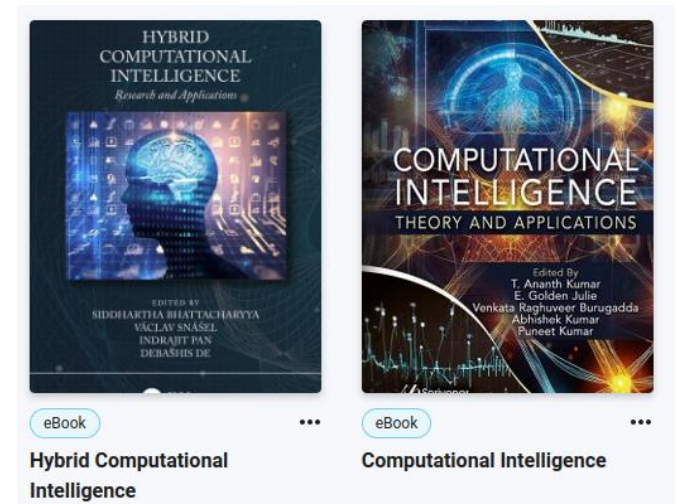
- A. Choosing affected tests
- B. Compiling source code
- C. Managing credentials
- D. Provisioning networks

Q6. What is the primary goal of sandboxing agentic CI workloads?

- A. Improve performance
- B. Enable parallelism
- C. Increase autonomy
- D. Contain failures

References

- [1] Available online, Introduction to Docker, Patel Jitendra, [https://www.academia.edu/11830102/Introduction to Docker](https://www.academia.edu/11830102/Introduction_to_Docker), last accessed 25/11/2026
- [2] Available online, Continuous Integration with Docker, Docker Documentation, <https://docs.docker.com/build/ci/>, last accessed 02/02/2026
- [3] Available online, Docker joins the Agentic AI Foundation, Docker Blog, <https://www.docker.com/blog/docker-joins-the-agentic-ai-foundation/>, last accessed 02/02/2026
- [4] Available online, From Code to Kubernetes: Leveraging Docker for Efficient CI/CD Workflows, Medium, <https://medium.com/@amitvsolutions/from-code-to-kubernetes-leveraging-docker-for-efficient-ci-cd-workflows-7745711af649>, last accessed 02/02/2026
- [5] Available online, A Review of Generative AI and DevOps Pipelines: CI/CD Automation, Preprints.org, <https://www.preprints.org/manuscript/202506.1040>, last accessed 02/02/2026





Nottingham Trent
University

Department of Computer Science

Summary

In this lecture you learnt:

- Motivation for modern CI
- Containerisation fundamentals
- Docker in CI pipelines
- Scalable distributed CI
- Kubernetes-based orchestration
- Observability and reliability
- Agentic AI in CI
- Security and governance

This lecture contributes to the following learning outcomes:

MLO2: Critically evaluate the strengths and weaknesses of different computational intelligence techniques for solving complex problems.

MLO4: Understand the ethical implications of using computational intelligence technologies and the potential impact on society.

MLO5: Apply computational intelligence techniques to solve real-world problems, demonstrating practical problem-solving skills.

MLO6: Develop proficiency in programming and using computational tools for implementing and evaluating intelligent systems.

MLO8: Communicate effectively the concepts, methodologies, and results of computational intelligence research to both technical and non-technical audiences.



Nottingham Trent
University

Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 5: Containerisation & Distributed Technologies for CI Workflows

Dr Pedro Machado pedro.machado@ntu.ac.uk